

ML Intern Lab: A Minimal Agentic Workflow for Reproducible Machine Learning Experiment Reports

Mukunda Rao Katta ORCID: <https://orcid.org/0009-0007-6071-3896> May 6, 2026

Abstract

Machine-learning teams increasingly use AI agents to read papers, propose experiments, run baselines, and draft reports. These workflows are useful, but they often blur together planning, execution, metrics, and narrative output in ways that are difficult to audit. This paper presents ML Intern Lab, a minimal agentic workflow for reproducible machine-learning experiment reports. The workflow turns an idea or paper note into an explicit experiment plan, runs a local baseline, writes machine-readable metrics, and generates a short model report. The reference implementation uses a zero-dependency Python runner and a tiny majority-class baseline experiment to demonstrate the pattern. The contribution is not a new model or benchmark. It is a compact reporting workflow that helps agentic ML assistants leave inspectable artifacts at each step, making their work easier to review, reproduce, and extend.

1. Introduction

AI agents are beginning to act as junior ML engineering assistants. They can summarize papers, suggest baseline experiments, generate training commands, inspect metrics, and draft reports. This is useful work, but it is easy for the resulting workflow to become a conversational transcript rather than a reproducible experiment. A team may know that an agent produced a result, but not exactly which assumptions, data file, baseline method, and metric path supported that result.

ML Intern Lab addresses that gap with a small artifact-first workflow. Instead of asking an agent to jump directly from an idea to a polished report, the workflow asks it to create and update inspectable files: a paper note, an experiment plan, a baseline run, a metrics file, and a model report. This makes the work legible to humans and repeatable by tools.

The method is deliberately small. A useful agentic ML workflow does not always need orchestration infrastructure, GPU scheduling, or a large tracking server. It often needs a clean first baseline, a stable report shape, and a clear path for the next experiment.

2. Design Goals

The workflow is built around four practical goals.

- Keep the experiment plan explicit enough for review before execution.
- Store metrics in a machine-readable file, not only in prose.
- Generate a human-readable report from the same run.
- Make the first baseline cheap enough that it can run locally in a few seconds.

These goals are meant for early-stage experimentation and educational settings. The workflow can also serve as a small test fixture for larger agent systems that claim to perform ML engineering tasks.

3. Workflow

The workflow has five steps:

Step	Artifact	Purpose
1	Paper note or idea note	Captures the hypothesis, source, and motivation.
2	Experiment plan JSON	Defines dataset path, target field, baseline method, and goal.
3	Baseline run	Executes the simplest useful model or heuristic.
4	<code>`metrics.json`</code>	Stores rows, target, predicted label, and accuracy.
5	<code>`model-report.md`</code>	Summarizes the goal, dataset, baseline, metric, and next steps.

This pattern gives an agent a clear trail to follow. It also gives a human reviewer several checkpoints where mistakes can be caught before a result is treated as meaningful.

4. Reference Implementation

The reference implementation is the ``ml-intern-lab`` Python package. It exposes a command-line entry point named ``ml-intern`` and currently includes a zero-dependency majority-class baseline runner. The baseline reads a CSV file, counts target labels, selects the majority label, computes accuracy, writes ``metrics.json``, and renders a Markdown model report.

The initial sample experiment uses seven rows and a categorical target named ``label``. The generated metrics are:

Field	Value
Experiment	<code>`0001-baseline`</code>
Baseline	<code>`majority-class`</code>
Rows	<code>`7`</code>
Target	<code>`label`</code>
Majority label	<code>`calendar`</code>
Accuracy	<code>`0.429`</code>

The small scale is intentional. The aim is to test the reporting workflow and artifact shape before adding more complex models.

5. Agentic Use Pattern

An AI assistant can use the workflow as a disciplined loop:

- Read a paper or product idea.
- Draft an experiment plan.
- Ask for review if the plan changes data, target, or metric assumptions.
- Run the baseline.
- Inspect the generated metrics.
- Draft the report from the metrics and plan.

- Propose the next experiment.

This division of labor keeps the agent from hiding important decisions inside fluent prose. It also gives maintainers a stable place to add validation, comparison, and approval gates.

6. Verification

The local package test suite was run on May 6, 2026 with:

```
PYTHONPATH=src python3 -m pytest -q
```

The result was:

```
1 passed in 0.01s
```

The generated sample metrics file reports a majority-class accuracy of `0.42857142857142855`, rounded to `0.429` in the model report.

7. Limitations

The current implementation is intentionally narrow. It does not train neural networks, split datasets, track multiple runs, or compare feature-based models. It also does not claim that majority-class accuracy is a strong baseline for real ML tasks. The baseline is a starting point for validating the workflow.

Future versions should add train-test splitting, simple feature models, repeated runs, dataset cards, and report sections for error analysis. These additions should preserve the current design principle: every agent action should leave a reviewable artifact.

8. Conclusion

ML Intern Lab offers a compact method for making agentic ML assistance more reproducible. By forcing a clean chain from idea to plan to metrics to report, it turns agent work into artifacts that can be inspected and rerun. The approach is small by design, but it addresses a practical need: teams need ML agents that produce evidence, not just confident summaries.

References

Katta, M. R. (2026). ML Intern Lab. Python package and experiment workflow.
<https://gitlab.com/mukunda.vjcs6-group/ml-intern-lab>

Hugging Face. (2026). ml-intern. Referenced as ecosystem context for agentic ML engineering workflows.
<https://github.com/huggingface/ml-intern>